

Lab 3: Blocking Cache

1 Introduction

The purpose of this lab was to gain hands-on experience with topics covered in lecture, including memory hierarchies, cache design, and finite-state-machine (FSM) control, by designing and evaluating two cache microarchitectures for the TinyRV2 processor. This lab emphasizes key computer architecture concepts from lecture such as hit and miss rates, average memory access latency (AMAL), and the trade-offs between hardware complexity, area, and performance. Our implementation followed an incremental development approach, beginning with a baseline cache design and then creating an alternative design with additional optimizations, building off the control/datapath split introduced in Lab 1. The baseline design implemented a direct-mapped, write-back, write-allocate cache. It handled memory requests by stalling the processor until the requested data was available, ensuring correctness but resulting in potential performance penalties due to cache misses. The alternative design implemented a two-way set-associative cache with a least-recently-used (LRU) replacement policy, allowing each cache index to store data in one of two locations. This design reduced conflict misses, improving cache hit rates and overall performance at the cost of additional control logic and hardware resources. Both designs used the TinyRV2 ISA for testing, leveraging instruction addresses to access the caches and validating functionality through directed and randomized test programs. Performance evaluations across our benchmark patterns showed that the alternative cache consistently reduced miss rates and AMAL for workloads with high conflict or temporal reuse: for example, `loop4` had a miss rate decrease from 97% to 96% (1% decrease) and an AMAL reduction from 45.58 to 38.22 (16% decrease), and `loop6` had a miss rate decrease from 100% to 25% (75% decrease) and an AMAL reduction from 48.58 to 14.80 (70% decrease), while simpler sequential workloads such as `loop1` and `loop5` exhibited minimal difference in both metrics. These results demonstrate that set-associative caches can provide substantial performance improvements for larger memory blocks, while direct-mapped caches remain advantageous for simpler, lower-latency applications.

2 Alternative Design

The alternative cache is a two-way set-associative, write-back, write-allocate design that maintains the baseline capacity and 16-B line size (256 B total) while adding only the minimal datapath and control for set associativity and LRU replacement. Externally, it matches the lab interface: the processor communicates via a 4 B val/rdy request/response stream, and the cache communicates with memory via a 16 B val/rdy line stream. The request `opaque` field is propagated end-to-end, and `cacheresp.test` is set to 01 on hits and 00 on misses so the harness can distinguish outcomes without inspecting the internals. Addressing is parameterized by the number of banks, $p_{\text{num_banks}} \in \{1, 4\}$. In the single-bank configuration, addresses are partitioned as `tag[31:7] | index[6:4] | word[3:2] | byte[1:0]`; in the four-bank configuration, `bank[5:4]` is inserted between index and word, and the tag is shortened to `tag[31:9]`. Bank bits are excluded from the set index so the same 2-way SA block can be reused inside a banked top level; the bank is added only when synthesizing full-line addresses for refills and evictions. For a miss, `mkaddr.refill` concatenates the current tag, set index, (and bank if present) with 4'b0000 to point to the start of the 16-B line. For evictions, the victim's tag forms `mkaddr.evict`, which is latched to decouple address generation from the memory handshake. The datapath has two parallel tag arrays ($25 \text{ b} \times 8 \text{ sets}$) and two parallel data arrays ($128 \text{ b} \times 8 \text{ sets}$) with combinational reads. Parallel tag comparisons generate `tag_match_way0` and `tag_match_way1`, which the controller qualifies with per-way valid bits to produce `hit0`, `hit1`, and `is_hit`. The selected hit way is latched during `TAG_CHECK` and enables only the winning way for the access. A victim-way select simultaneously targets the readback of an about-to-be-evicted line for refills. The read path uses a compact mux chain: first, a way mux selects the hit line; second, a victim mux exposes the miss target; third, a read-source mux can bypass the arrays to source directly from memory immediately after a refill. A final 4:1 word-select mux extracts the requested 32-B word using the address word offset. The write path replicates the CPU's 32-B data across the 128-B line and uses a per-word write-enable decoder so only the addressed word is updated on hits, while refills write the full `memresp` line to preserve write-allocate semantics without adding datapath complexity.

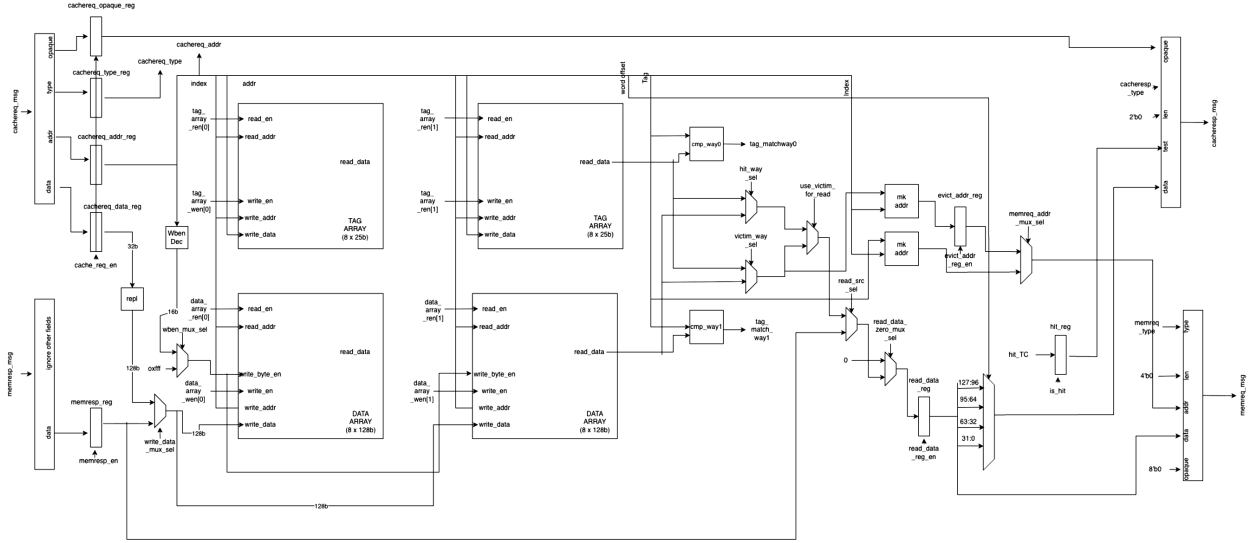


Figure 1: Alternative Design DataPath

The control logic is table-driven and extends the baseline FSM with explicit two-way behavior and meta-data management. Each request first enters TAG_CHECK. An INIT request proceeds directly to INIT_DATA_ACCESS to seed the chosen way and clear its dirty bit before returning to WAIT. On a hit, the FSM branches to READ_DATA_ACCESS or WRITE_DATA_ACCESS, then to WAIT to complete the response handshake. On a miss, a victim is selected: an invalid way is preferred, otherwise a per-set 1-bit LRU register determines the least-recently-used way ($0 \Rightarrow \text{way0}$, $1 \Rightarrow \text{way1}$). If the victim is dirty, the FSM enters EVICT_PREPARE to read the line, issues a write in EVICT_REQUEST, and waits in EVICT_WAIT; clean victims skip directly to REFILL_REQUEST and REFILL_WAIT. Upon refill arrival, REFILL_UPDATE writes the line into the victim way, marks it valid, sets the dirty bit according to write intent, and either responds immediately or proceeds to the write access. A “refill fast-path” allows the controller to select the response data directly from the incoming memory line or just-written victim way, avoiding an extra array turn-around and saving a cycle; `read_src_sel` and `use_victim_for_read` coordinate this bypass. Val/rdy handshakes are asserted only when corresponding to active transactions: `cache2mem_reqstream_val` for refills/evictions, `memreq_type` to distinguish them, and `cache2mem_respstream_rdy` in wait states. The response path preserves `opaque` and copies the request `type`, while `cacheresp.test` indicates hit/miss outcomes. Per-set metadata resides in lightweight, resettable regfiles: valid and dirty bits per way and a single LRU bit per set. These are updated in three situations: INIT sets the chosen way valid and clears its dirty bit; write hits set the dirty bit on the hit way; and REFILL_UPDATE sets the victim way valid and dirty depending on the request. The LRU bit is updated on every access so the non-accessed way becomes LRU, ensuring consistent replacement. Together, the FSM and metadata guarantee correct write-back/write-allocate behavior, line-granular refills and evictions, and partial-word writes on hits. Relative to the baseline, the datapath adds a second tag/data array, an additional comparator, hit/victim way muxing, a read-source mux for the fast-path, and an evict-address register to form `mkaddr.evict`. Control adds per-set valid/dirty regfiles, a 1-bit LRU regfile, latches for `hit_way.q`, `victim_way.q`, and `resp_is_hit`, plus extended control-table rows to drive arrays, handshakes, and metadata updates during EVICT and REFILL. Functionally, the design fulfills the lab requirements for a two-way set-associative cache with LRU replacement and write-back/write-allocate policy, while keeping control simple and enabling single-cycle tag checks using combinational SRAM reads.

Testing Methodology

We used the provided testing framework along with directed, randomized, and integration tests to verify both the baseline and alternative cache designs. We followed an incremental and test-driven approach, implementing and verifying each cache operation such as read hits, write hits, read misses, and write misses

before continuing to the next. These unit-level tests ensured that each operation produced correct data values, updated valid bits and tags properly, and asserted the correct ready/valid handshake signals. For example, we verified that a read hit returned the correct data immediately, a write hit updated the cache line without requiring a memory access, and both read and write misses correctly generated memory requests and refilled cache lines from memory. Beyond verifying individual operations, our directed tests were designed to exercise different aspects of the cache’s functionality and edge cases. These included filling an entire cache line to verify correct word offset handling and alignment, triggering conflict misses by repeatedly accessing addresses that map to the same index to test proper tag comparison and eviction, writing to all words within a line to confirm correct byte-enable and writeback logic, and testing capacity misses by exceeding the total number of cache lines to ensure proper replacement behavior. We also verified that valid and dirty bits were correctly updated across hits, misses, and refills, confirming accurate state transitions in both the datapath and control logic. Functional correctness of the direct-mapped cache was further validated using the dmap test suite, covering conflict scenarios, thrashing, sequential fills, wraparound, stride-based accesses, dirty line evictions, and accesses to different word offsets within the same line. For the set-associative alternative design, we created additional directed tests to validate way-selection and eviction behavior. These tests included repeatedly writing to the same set to confirm that only the least-recently-used line was replaced, triggering alternating conflict misses across both ways to verify proper tag matching and replacement decisions, and ensuring that dirty lines were written back to memory upon eviction. These cases confirmed that our LRU logic correctly tracked recent accesses and maintained consistent data coherence between cache and memory. To ensure functional correctness, we compared all test results against a functional-level Python reference model. The Python model implemented the same cache organization using arrays for tags, data, and main memory, and deterministically computed hits, misses, and evictions independent of hardware timing. This model served as a golden reference for verifying the Verilog cache’s behavior. When mismatches occurred, we used line traces and waveform inspection to pinpoint whether the issue originated in the datapath, control FSM, or replacement policy. Directed tests also covered corner cases such as repeatedly writing to the same address, filling the entire cache sequentially, and testing index wraparound to confirm correct handling of all addressing patterns. We also developed randomized value and timing tests to evaluate the cache under less predictable and more realistic workloads. We generated random sequences of addresses and data, restricted to approximately 250 unique locations to increase collision frequency and thoroughly exercise the replacement logic. We also introduced random stalls and delays in the source, sink, and memory interfaces to ensure that our design maintained correctness and responsiveness under backpressure and varied timing conditions. These tests confirmed that our cache correctly handled concurrent operations, maintained stability under unpredictable timing, and consistently produced valid outputs.

Table 1: Summary of Cache Test Suite

Cache Type of Test	Number of Tests
Random	6
Direct-Mapped (DM)	10
Set-Associative (SA)	9
Generic	43
Total	68

3 Evaluation

We evaluated both the baseline and alternative cache designs using the provided memory access patterns `loop1`, `loop2`, and `loop3`, along with three additional patterns `loop4`, `loop5`, and `loop6` that we created to introduce a variety of spatial locality, temporal locality, and mixed read/write behavior. These patterns cover a wide spectrum of memory access scenarios, from simple sequential array traversal to irregular, strided, and randomized accesses with bursts of writes.

Performance Results. In the simplest case, `loop1` performs a sequential traversal of a single array. Both caches exhibit similar performance for this pattern because every first access to a memory block results in a compulsory miss. Once the cache lines are loaded, conflict misses are negligible, so the baseline and

alternative designs behave equivalently. Here, the baseline cache has the advantage of lower hardware area and slightly reduced dynamic power per access due to its simpler control logic. For workloads that are mostly sequential or highly localized, the baseline cache may be preferred because the alternative design provides no significant benefit.

The differences between the caches become apparent in multi-array patterns, such as `loop2` and `loop3`, which access multiple arrays that map to overlapping cache sets. The baseline cache experiences frequent conflict misses because new accesses evict lines that will soon be needed again. The alternative cache, with improved associativity and replacement policies, retains these lines more effectively, resulting in lower miss rates and reduced average memory access latency (AMAL). In these scenarios, the alternative cache is generally preferred, especially when execution time and memory efficiency are critical, whereas the baseline may still be acceptable if area or simplicity is prioritized. In `loop4`, which performs a vector-vector addition (VVADD) with irregular stride and a randomized read/write order, and `loop5`, which accesses a set of random addresses with intermittent bursts of writes, the cache's ability to retain intermittently used data is challenged. The baseline cache evicts useful lines more often, causing repeated main memory accesses, while the alternative cache better preserves these lines, reducing stall cycles and improving overall execution time. For unpredictable access patterns or intermittent writes, the alternative design clearly outperforms the baseline, though the baseline may still be chosen when minimizing hardware complexity or dynamic power per access is important. `Loop6` emphasizes temporal locality by revisiting previously accessed data multiple times in multiple passes, simulating repeated updates to arrays that intentionally cause cache conflicts. The baseline cache repeatedly replaces lines that will soon be reused, leading to extra memory refills. In contrast, the alternative cache retains these lines across iterations, leveraging temporal locality and significantly reducing stall cycles. Here, the alternative cache is strongly favored due to its ability to exploit temporal reuse, while the baseline may be sufficient only if the workload rarely revisits data.

Overall, the alternative cache consistently completes execution in fewer cycles across all patterns with significant conflict potential or reuse, not due to a faster clock, but because it minimizes costly memory accesses. The baseline cache remains adequate for simple, highly localized workloads, but the alternative design is clearly superior for scenarios where hit rate and memory efficiency are critical.

Table 2: Cache AMAL Statistics for Base and Alternate Implementations

Loop	Impl	Num Cycles	Num Requests	Num Misses	Miss Rate %	AMAL
loop1	base	984	100	25	0.25	9.84
loop1	alt	984	100	25	0.25	9.84
loop2	base	5409	200	200	1.00	27.05
loop2	alt	1959	200	50	0.25	9.79
loop3	base	6384	300	225	0.75	21.28
loop3	alt	8109	300	300	1.00	27.03
loop4	base	13675	300	290	0.97	45.58
loop4	alt	11467	300	287	0.96	38.22
loop5	base	2674	120	61	0.51	22.28
loop5	alt	2674	120	61	0.51	22.28
loop6	base	24873	512	512	1.00	48.58
loop6	alt	7577	512	128	0.25	14.80

Cycle Time Analysis. While both caches operate under the same simulated clock frequency, total execution time depends heavily on the number of cache misses and subsequent memory refills. The baseline cache stalls more frequently under conflict-heavy and mixed-access workloads, each stall incurring a full main memory latency of 20 cycles. By reducing the number of misses, the alternative cache minimizes these stalls. Consequently, even though per-cycle delays are similar, the alternative design achieves faster total execution across all patterns.

Area Analysis. The baseline cache occupies less hardware area due to its simpler control logic and minimal replacement tracking. The alternative cache adds tag comparators, replacement state bits, and slightly more complex control logic. This results in a modest area increase, but the additional structures are small relative to the total cache storage. The performance gains in hit rate and reduced memory stalls

justify this area overhead. When area is a critical constraint and workloads are simple, the baseline cache may still be preferred.

Energy Analysis. The alternative cache consumes slightly more dynamic power per access due to its additional logic. However, it reduces the total number of main memory accesses, which are significantly more energy-intensive. The baseline cache, with its higher miss rate, performs more long-latency memory fetches, increasing energy consumption. Therefore, when considering total energy per workload, the alternative cache is more energy efficient despite its slightly higher per-access cost. For very small, predictable workloads, the baseline may still be sufficient if energy per access is the main concern.

Summary. In summary, the alternative cache design outperforms the baseline across most workloads that involve conflict potential, temporal reuse, or irregular access patterns. The modest area and dynamic energy overheads are outweighed by the benefits of reduced miss rates, lower execution cycles, and improved overall energy efficiency. The baseline cache remains adequate for simple, highly localized workloads, where low area and minimal per-access energy are prioritized. Choosing between the two designs depends on the expected workload: for predictable, sequential access, the baseline is sufficient; for multi-array, irregular, or temporally reusable access patterns, the alternative cache provides clear advantages.

4 Conclusion

This lab tasked us with implementing and evaluating two cache microarchitectures: a direct-mapped baseline and a two-way set-associative (2-way SA) alternative, both using write-back, write-allocate policies. Our goals were to verify functional correctness and quantitatively evaluate performance using cycles, misses, miss rate, and average memory access latency (AMAL). The evaluation reveals that the alternative cache provides significant benefits for workloads prone to conflicts or with high temporal reuse. For example, in **loop2**, which accesses multiple arrays that map to overlapping cache sets, the baseline suffers from frequent conflict misses. The alternative cache reduces the miss rate from 100% to 25% (75%), lowers AMAL from 27.05 to 9.79 (63.8%), and decreases total cycles from 5409 to 1959 (63.7%). Similarly, **loop6**, designed to revisit previously accessed data multiple times, shows a drop in miss rate from 100% to 25% (75%), AMAL from 48.58 to 14.80 (-69.5%), and cycles from 24873 to 7577 (-69.5%), demonstrating the alternative cache's ability to exploit temporal locality and reduce memory stalls. For workloads with minimal conflicts, such as **loop1**, both caches perform identically (miss rate 25%, AMAL 9.84, cycles 984), indicating that the added complexity of the alternative design does not impose overhead when associativity is not needed. One minor regression occurs in **loop3**, where the alternative cache shows a higher miss rate (75% to 100%) and increased cycles (6384 to 8109, +27%) due to way-thrashing for specific address patterns; this can be mitigated with small modifications, such as a victim buffer or adjusted LRU updates. Overall, the two-way set-associative cache clearly reduces miss rates, AMAL, and cycles for workloads with conflict potential or temporal reuse, while maintaining parity with the baseline on simple sequential workloads. The alternative cache achieves up to 70% reduction in AMAL and execution cycles in the most challenging benchmarks. These results show that associativity is most beneficial for multi-array or irregular access patterns, while the simpler baseline cache suffices for predictable, sequential memory accesses. Consequently, for future designs, a two-way set-associative cache should be favored when conflicts or reuse are expected, but the baseline remains suitable for small, simple workloads where area and power savings are prioritized.

5 Work Distribution

Both team members made significant contributions to the design, implementation, and testing of the Lab 3 cache processor. Md primarily implemented and tested the baseline cache, developing its datapath and control logic, managing read/write hits and misses, and creating directed and randomized tests covering edge cases such as filled cache lines, conflict misses, and capacity misses. Correctness was verified using Python-based functional simulations of tags, data arrays, memory, and hit/miss behavior. Jamayne focused on the alternative set-associative cache, implementing datapath and control logic for way selection, evictions, and conflict resolution, and developing directed and randomized tests specific to this design, including LRU replacement and varied access patterns. Verification was also performed with Python functional models for the two-way tag arrays. Both collaborated closely on debugging, design reviews, and integration testing

to ensure consistency and correctness across designs. Generative AI tools were used in three ways: for coding, AI assisted in identifying and correcting Verilog issues such as missing signal declarations, incorrect module instantiations, mismatched signal widths, and undefined wires; for testing, AI helped brainstorm and partially automate directed and randomized test cases to improve coverage and address edge cases; and for report preparation, AI refined phrasing, grammar, clarity, and presentation of technical concepts. Md documented the baseline testing methodology and authored the baseline design and evaluation sections, while Jamayne authored the alternative design, evaluation, and comparisons sections.

6 Appendix

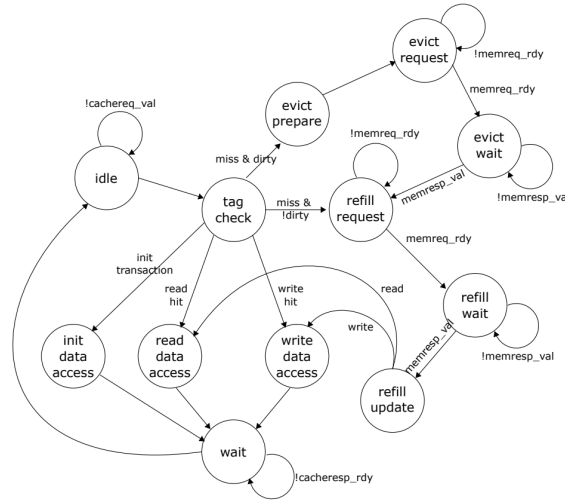


Figure 2: Baseline FSM Control Unit

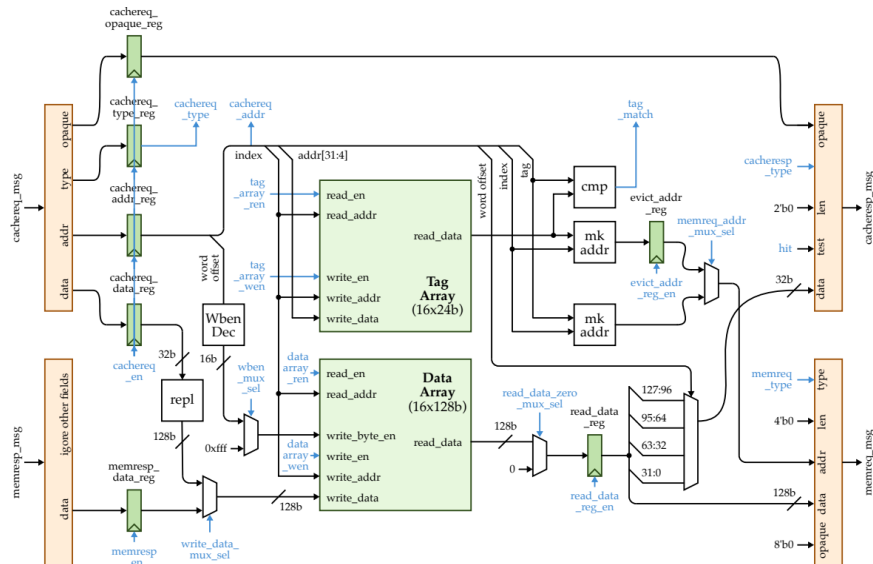


Figure 3: Baseline Data Path